Lab 1 & Lab 2 Technical Report

Title: Linux Server Security Assessment and Hardening

Domain: Security Engineering, Linux Hardening, Infrastructure Defense

Environment: AWS EC2 - Ubuntu 24.04.4 LTS

Scope: Mega-Lab 1 (Security Assessment & Threat Discovery) and Mega-Lab 2 (Firewalling, SSH Security & Linux Hardening)

> **Purpose:** This document consolidates the complete technical narrative of Lab 1 and Lab 2, including assessment logic, command purpose, field evidence, the custom baseline script, findings, threat modeling, hardening actions, and validation results. The goal is to make the document self-sufficient so it can stand as portfolio-grade evidence even without additional screenshots.

Figure 1. Module 03 topic map used as the conceptual frame for the project.

1. Executive Summary

This project documents the end-to-end assessment and hardening of an AWS EC2 Ubuntu server. The work started with discovery and evidence collection: identifying the operating system, kernel, local accounts, sudo privileges, SSH configuration, listening ports, running services, firewall state, package update posture, and failed authentication activity. That

baseline then informed a controlled hardening phase that reduced unnecessary exposure while preserving access and auditability.

The work followed a repeatable engineering pattern:

```
Threat -> Control -> Validation
```

Instead of changing the system blindly, each hardening step was justified by a concrete finding and then validated using direct technical evidence.

2. Project Objectives

- Establish a reproducible baseline assessment process for a Linux server.
- Identify security weaknesses that can be discovered from host-level inspection.
- Create a reusable Bash script to automate evidence collection.
- Evaluate exposed services, identity risks, and SSH security posture.
- Apply targeted hardening controls to reduce attack surface.
- Validate all changes with before-and-after checks.
- Produce documentation suitable for a professional GitHub portfolio.

3. Environment and Operational Context

Component	Value
Cloud Provider	AWS
Compute Service	EC2
Instance Platform	Ubuntu 24.04.4 LTS
Kernel	Linux 7.0.0-1009-aws
Hardware Model	t3.micro
Architecture	x86_64
Remote Management	SSH
Host Firewall	UFW
SSH Protection	Fail2Ban
Additional Public Service Discovered	Nginx on port 80

4. Module Coverage

The practical work in this report maps to the following Module 03 topics:

- Security Foundations & Risk Management
- Threat Modeling & Attack Lifecycle

- Network and Availability Threats
- Firewalling & Network Defense
- Cryptography, PKI & SSH Security
- Linux Hardening & Vulnerability Management
- System Auditing, Logging & SIEM preparation
- Log-Based Abuse Mitigation readiness

This report specifically covers the first two practical phases:

Lab	Focus	Status
Lab 1	Security Assessment, Asset Discovery, Threat Discovery	Completed
Lab 2	Linux Hardening, SSH Hardening, Service Reduction, Firewall Cleanup	Completed

5. Methodology

5.1 Engineering Logic

Each control in the hardening phase was tied to a prior observation. The methodology intentionally avoided unstructured trial-and-error. The workflow was:

```
System Identification
-> User and Privilege Analysis
-> SSH Configuration Review
-> Network Exposure Review
-> Service Inventory
-> Firewall Review
-> Authentication Signal Review
-> Baseline Automation
-> Threat Modeling
-> Controlled Hardening
-> Validation
```

5.2 Threat - Control - Validation Model

Threat	Control	Validation
Unnecessary network-exposed service	Stop service and remove firewall rule	Confirm service inactive and port not exposed
Unused local account with active password	Lock account	Check password state changes from `P` to `L`
Unsafe SSH feature enabled	Disable X11 forwarding	Verify effective SSH configuration with `sshd -T`
Weak file ownership/ permission model	Apply restrictive permissions and correct ownership	Re-list target directory and confirm exact mode and owner

6. Lab 1 - Baseline Security Assessment

6.1 Goal

The objective of Lab 1 was to understand the actual state of the server before making defensive changes. This phase answers: what exists, what is exposed, what is privileged, and what is likely risky.

6.2 System Identity Collection

The first commands were:

```
uname -a
hostnamectl
```

These commands were used to identify the operating system, kernel line, architecture, virtualization context, and EC2 hardware profile.

Observed evidence:

```
Operating System: Ubuntu 24.04.4 LTS
Kernel: Linux 7.0.0-1009-aws
Architecture: x86-64
Virtualization: amazon
Hardware Vendor: Amazon EC2
Hardware Model: t3.micro
```

This confirmed that the target was a cloud-hosted Ubuntu system on AWS and not a local virtual machine or container.

6.3 Current User and Privilege Review

To understand administrative capability, the following commands were used:

```
whoami
id
sudo -l
```

Observed evidence:

```
ubuntu
uid=1000(ubuntu) gid=1000(ubuntu) groups=1000(ubuntu),4(adm),24(cdrom),27(sudo),
30(dip),105(lxd)

User ubuntu may run the following commands on ip-172-31-42-76:
    (ALL : ALL) ALL
    (ALL) NOPASSWD: ALL
```

This showed that the `ubuntu` account was a full administrative identity with passwordless sudo capability. From a cloud usability perspective this is common. From a security

perspective it is a critical identity because any compromise of the account or its SSH key could immediately lead to root-level control.

6.4 Local Account Inventory

The local account list was enumerated with:

```
awk -F: '$3 >= 1000 {print $1, $3, $6, $7}' /etc/passwd
```

Observed evidence:

```
nobody 65534 /nonexistent /usr/sbin/nologin
ubuntu 1000 /home/ubuntu /bin/bash
dev-user 1001 /home/dev-user /bin/bash
```

This revealed the presence of an additional local account, `dev-user`, which required further review.

Follow-up commands:

```
sudo groups dev-user
sudo passwd -S dev-user
sudo -l -U dev-user
```

Observed evidence:

```
dev-user : dev-user users
dev-user P 2026-07-30 0 99999 7 -1

User dev-user may run the following commands on ip-172-31-42-76:
    (ALL) NOPASSWD: /usr/bin/systemctl status *
```

The account was not a full administrator, but it had an active password and limited passwordless sudo capability. This made it a non-trivial identity from a security standpoint.

6.5 SSH Security Review

The effective SSH configuration was checked using:

```
sudo sshd -T | grep -E '^(permitrootlogin|passwordauthentication|
pubkeyauthentication|kbdinteractiveauthentication|maxauthtries|allowusers|
permitemptypasswords|x11forwarding|port)'
```

Observed evidence:

```
port 22
maxauthtries 3
permitrootlogin no
pubkeyauthentication yes
passwordauthentication no
kbdinteractiveauthentication no
```

```
x11forwarding yes
permitemptypasswords no
```

Several secure defaults were already present: direct root SSH login was disabled, password authentication was disabled, keyboard-interactive authentication was disabled, and public-key authentication was enabled. However, `X11Forwarding` was enabled on a headless EC2 server, which was unnecessary and broadened the SSH feature set without operational value.

6.6 Service and Network Exposure Review

Listening services were inspected with:

```
sudo ss -tlnp
```

Key observed exposure included:

```
0.0.0.0:80     nginx
0.0.0.0:22     sshd
[::]:80        nginx
[::]:22        sshd
```

This showed that both SSH and Nginx were reachable on all network interfaces. The presence of Nginx on port 80 was especially important because no intentional production web application had been defined for this lab stage.

To confirm local web-server behavior, the following command was used:

```
curl -I localhost
```

Observed evidence:

```
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Date: Wed, 05 Aug 2026 10:54:03 GMT
Content-Type: text/html
Content-Length: 615
```

This proved that Nginx was actively serving content rather than merely installed.

6.7 Firewall Review

The host firewall rules were checked with:

```
sudo ufw status numbered
```

Observed evidence before cleanup:

```
Status: active
```

```
[ 1] 22/tcp                     ALLOW IN    Anywhere
[ 2] 80/tcp                     ALLOW IN    Anywhere
[ 3] 22/tcp (v6)                ALLOW IN    Anywhere (v6)
[ 4] 80/tcp (v6)                ALLOW IN    Anywhere (v6)
```

The inbound default-deny posture was useful, but the explicit HTTP allow rule on port 80 still exposed the Nginx service to the internet.

6.8 Authentication Signal Review

Authentication-related evidence showed that the internet-exposed SSH service was already being scanned. Relevant examples included invalid-user or malformed connection attempts recorded in the authentication logs. In addition, the Fail2Ban status later showed SSH-related activity had already been seen by the jail logic.

6.9 Why a Baseline Script Was Created

At this point, manually repeating each assessment command would have been error-prone and time-consuming. A read-only automation script was created to collect the most important security signals in one run and write them to a timestamped report.

7. Custom Script - `ec2_security_baseline_check.sh`

7.1 Purpose of the Script

The script was designed to automate host-level evidence collection without changing the system state. It acts as a repeatable baseline assessment tool for Linux security reviews.

- It inventories identity and privilege information.
- It checks the effective SSH posture.
- It lists listening services and running units.
- It checks firewall state.
- It looks for pending security updates.
- It reviews recent failed SSH-related log activity.
- It records the information in a timestamped report.

7.2 Full Script

```
#!/bin/bash

# ============================================================
# EC2 Security Baseline Assessment Script
# Version : 1.0
# Purpose : Read-only security inventory - no changes made
# ============================================================

REPORT="$HOME/baseline_report_$(date +%Y%m%d_%H%M%S).txt"

log() {
    echo -e "$1" | tee -a "$REPORT"
```

```
}

log "============================================================"
log " EC2 SECURITY BASELINE REPORT"
log " Generated : $(date)"
log " Hostname  : $(hostname)"
log "============================================================"

log "\n=== 1. SYSTEM IDENTITY ==="
hostnamectl | tee -a "$REPORT"

log "\n=== 2. CURRENT USER & PRIVILEGES ==="
log "User    : $(whoami)"
log "ID info : $(id)"

log "\n=== 3. SUDO PERMISSIONS - CURRENT USER ==="
sudo -l 2>&1 | tee -a "$REPORT"

log "\n=== 4. LOCAL USER ACCOUNTS (UID >= 1000) ==="
awk -F: '$3 >= 1000 {
    printf "%-15s UID=%-6s HOME=%-20s SHELL=%s\n",
    $1,$3,$6,$7
}' /etc/passwd | tee -a "$REPORT"

log "\n=== 5. SUDO GROUP MEMBERS ==="
getent group sudo | tee -a "$REPORT"

log "\n=== 6. SUDO PERMISSIONS - ALL LOCAL USERS ==="
awk -F: '$3 >= 1000 && $3 < 65534 {print $1}' /etc/passwd |
while read user; do
    log "\n--- $user ---"
    sudo -l -U "$user" 2>&1 | tee -a "$REPORT"
done

log "\n=== 7. SSH EFFECTIVE CONFIGURATION ==="
sudo sshd -T |
grep -E '^(permitrootlogin|passwordauthentication|pubkeyauthentication|
kbdinteractiveauthentication|maxauthtries|allowusers|permitemptypasswords|
x11forwarding|port) ' |
tee -a "$REPORT"

log "\n=== 8. SSH SERVICE STATUS ==="
log "Enabled : $(systemctl is-enabled ssh 2>/dev/null || systemctl is-enabled
sshd 2>/dev/null)"
log "Active  : $(systemctl is-active ssh 2>/dev/null || systemctl is-active sshd
2>/dev/null)"

log "\n=== 9. LISTENING PORTS & SERVICES ==="
sudo ss -tlnp | tee -a "$REPORT"

log "\n=== 10. RUNNING SERVICES ==="
systemctl list-units \
    --type=service \
    --state=running \
    --no-pager | tee -a "$REPORT"

log "\n=== 11. FIREWALL STATUS ==="
```

```
sudo ufw status verbose 2>&1 | tee -a "$REPORT"

log "\n=== 12. PENDING SECURITY UPDATES ==="
apt list --upgradable 2>/dev/null |
grep -i security |
tee -a "$REPORT"

log "\n=== 13. FAILED SSH LOGIN ATTEMPTS (last 20) ==="
sudo grep -i 'failed\|invalid' /var/log/auth.log 2>/dev/null |
tail -n 20 |
tee -a "$REPORT"

log "\n=== 14. RECENT SUCCESSFUL LOGINS ==="
last -n 10 | tee -a "$REPORT"

log "\n=== 15. DISK USAGE ==="
df -h | tee -a "$REPORT"

log "\n============================================================"
log " ASSESSMENT COMPLETE"
log " Report saved to: $REPORT"
log "============================================================"
```

7.3 Script Design Notes

- The script is intentionally read-only.
- It combines inventory, exposure review, and operational evidence collection.
- It uses `tee -a` so output is visible in the terminal and persisted in the report file.
- Because the script was executed with `sudo`, `$HOME` resolved to `/root`, which caused reports to be written under the root home directory.

8. Threat Modeling and Risk Assessment

8.1 Asset Inventory

Asset	Description	Security Relevance
EC2 instance	Cloud-hosted Ubuntu server	High
SSH service	Primary remote management channel	Critical
`ubuntu` account	Main admin account	Critical
`dev-user` account	Secondary local identity	Medium to High
UFW rules	Host-level exposure control	Critical
Nginx service	Public HTTP service	Medium
SSH configuration	Remote access control set	Critical
Authentication logs	Detection evidence for login activity	High

Kernel and packages	Core execution layer	Critical

8.2 Risk Register

ID	Finding	Threat	Control	Validation
R-01	Public SSH exposure on port 22	Brute-force attempts, scanning, credential abuse	Retain key-based access, keep Fail2Ban active, keep password auth disabled	Check effective SSH config and Fail2Ban jail status
R-02	Nginx listening on port 80	Unnecessary attack surface, version leakage, future web exploitation path	Stop Nginx and remove firewall allow rule	Verify service inactive and only 22 remains allowed
R-03	`X11Forwarding yes`	Unnecessary SSH feature exposure	Disable X11 forwarding	Re-check with `sshd -T`
R-04	`dev-user` had active password	Additional authentication entry point	Lock account	Confirm password state changes to locked
R-05	`/home/dev-user/.ssh` had weak permissions and wrong owner	Key tampering or unauthorized access to SSH material	Apply `chmod 700` and correct ownership	Re-list directory and confirm exact ownership and mode
R-06	`ModemManager` running on EC2	Unnecessary service footprint	Stop and disable service	Check service active/enabled state
R-07	Pending kernel-related package updates appeared later	Known vulnerabilities remain if patches are not applied	Apply updates and verify package state	Check post-update package state and reboot where required

8.3 STRIDE Mapping

STRIDE Category	Example in this Lab	Mitigation
Spoofing	Unauthorized SSH authentication attempts against public SSH	Key-based authentication, password auth disabled, Fail2Ban
Tampering	Weak ownership and permissions in the SSH directory	Restrictive mode and correct ownership
Repudiation		

	Need for evidence of login or failed-access attempts	Authentication log review and repeatable baseline reports
Information Disclosure	Default Nginx service revealing HTTP service presence and version family	Disable service and remove public exposure
Denial of Service	Repeated abusive connection attempts against SSH	Fail2Ban and limited authentication attempts
Elevation of Privilege	Administrative account abuse or exploitable outdated components	Least privilege review and patch management

9. Lab 2 - Hardening Actions

9.1 Disable X11 Forwarding

First, the active configuration line was located:

```
sudo grep -n "X11Forwarding" /etc/ssh/sshd_config
```

Observed evidence:

```
99:X11Forwarding yes
128:#   X11Forwarding no
```

The active line was modified and the SSH configuration was syntax-checked before reload:

```
sudo sed -i '99s/yes/no/' /etc/ssh/sshd_config
sudo sshd -t && sudo systemctl reload ssh
sudo sshd -T | grep x11forwarding
```

Validation result:

```
x11forwarding no
```

This is a strong example of correct defensive change management: identify the line, change the line, test the daemon configuration, reload safely, and verify the effective runtime setting.

9.2 Disable Unnecessary ModemManager Service

Because the EC2 instance had no operational need for modem management, the service was stopped and disabled:

```
sudo systemctl stop ModemManager && sudo systemctl disable ModemManager
```

Observed output:

```
Removed "/etc/systemd/system/dbus-org.freedesktop.ModemManager1.service".
Removed "/etc/systemd/system/multi-user.target.wants/ModemManager.service".
```

Validation used:

```
systemctl is-active ModemManager
systemctl is-enabled ModemManager
```

Observed state included:

```
inactive
```

This reduced the footprint of non-essential software on the host.

9.3 Lock the `dev-user` Account

The local account was locked instead of deleted:

```
sudo usermod -L dev-user
sudo passwd -S dev-user
```

Validation result:

```
dev-user L 2026-07-30 0 99999 7 -1
```

The state changed from `P` to `L`, confirming that the password had been locked. This preserves ownership and traceability while removing straightforward password-based use of the account.

9.4 Correct `.ssh` Directory Ownership and Permissions

The SSH directory was inspected:

```
sudo ls -la /home/dev-user/.ssh/ 2>/dev/null || echo "No .ssh directory found"
```

Observed evidence:

```
total 8
drwxrwxrwx 2 root     root     4096 Jul 30 12:24 .
drwxr-x--- 3 dev-user dev-user 4096 Jul 30 12:24 ..
```

The corrective commands used were:

```
sudo chmod 700 /home/dev-user/.ssh
sudo chown dev-user:dev-user /home/dev-user/.ssh
```

Validation:

```
sudo ls -la /home/dev-user/ | grep .ssh
```

Observed evidence:

```
drwx------ 2 dev-user dev-user 4096 Jul 30 12:24 .ssh
```

This corrected both the privilege model and the ownership model.

9.5 Remove Unused Nginx Exposure

Nginx had been confirmed live through local HTTP response testing. Because no intentional web workload existed, the service was stopped and disabled:

```
sudo systemctl stop nginx && sudo systemctl disable nginx
```

The public firewall rule was then removed:

```
sudo ufw delete allow 80/tcp
```

Observed output:

```
Rule deleted
Rule deleted (v6)
```

Validation used:

```
sudo ufw status numbered
systemctl is-active nginx
```

Observed evidence:

```
Status: active

[ 1] 22/tcp                     ALLOW IN    Anywhere
[ 2] 22/tcp (v6)                ALLOW IN    Anywhere (v6)

inactive
```

Only SSH remained intentionally reachable through the host firewall.

9.6 Review Fail2Ban State

The SSH jail status was reviewed:

```
sudo fail2ban-client status sshd
```

Observed evidence:

```
Status for the jail: sshd
|- Filter
|  |- Currently failed: 0
|  |- Total failed:     1
|  `- Journal matches:  _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
   |- Currently banned: 0
   |- Total banned:     0
   `- Banned IP list:
```

This showed that the defense mechanism was active and had already seen SSH-related events, even though the threshold for banning had not been met during the observed period.

10. Validation and Before/After Comparison

10.1 Re-running the Baseline Script

After hardening, the same script was executed again:

```
sudo ~/ec2_security_baseline_check.sh
```

This is an important methodological detail. Security changes should be validated by re-measurement, not by assumption.

10.2 Key Before/After State Changes

Area	Before	After
SSH X11 Forwarding	`x11forwarding yes`	`x11forwarding no`
HTTP exposure	Nginx exposed on `0.0.0.0:80` and `[::]:80`	Port 80 no longer allowed through UFW and Nginx inactive
`dev-user` password state	`P` (active password)	`L` (locked)
`/home/dev-user/.ssh`	`drwxrwxrwx root root`	`drwx------ dev-user dev-user`
ModemManager	Running / enabled	Stopped / disabled
Firewall rule set	22 and 80 allowed	Only 22 allowed

10.3 Post-Hardening Evidence

The baseline rerun confirmed that the major hardening actions were effective. The firewall remained active, the HTTP allow rule was removed, unnecessary services were no longer active, and the SSH configuration reflected the X11 forwarding change.

A later check also showed that kernel-related updates had to be reviewed again, demonstrating a practical truth of vulnerability management: update state is time-sensitive and must be monitored continuously.

11. Technical Interpretation of Findings

11.1 Why Nginx Was a Real Security Finding

A service is not safe merely because it is a default installation or returns a simple page. If a publicly reachable service has no business purpose, it is unnecessary attack surface. In this case, Nginx exposed the host to HTTP probing, potential version-family fingerprinting, and any future web-based misconfiguration risk.

11.2 Why the `dev-user` Account Mattered

Even limited or undocumented local accounts matter because they expand the authentication surface. An attacker does not need full sudo to benefit from a second account. The account was therefore treated as a meaningful identity risk and locked.

11.3 Why Effective SSH Configuration Matters More Than Reading One File

The command `sshd -T` shows the effective configuration that SSH would apply. This is more trustworthy than only reading the configuration file because actual daemon behavior can be influenced by include directives or additional config paths.

11.4 Why Permission Fixes Were Important

The `.ssh` directory is security-critical because it may contain authentication material. World-writable permissions and incorrect ownership create conditions for tampering, key misuse, or trust-model breakdown.

12. Lessons Learned

- Inventory must come before hardening.
- Effective configuration should be validated through runtime-aware commands.
- Unused services are risk even if they appear harmless.
- Identity review is as important as port review.
- Permissions and ownership are part of access control, not just housekeeping.
- Security validation requires a before-and-after evidence trail.
- Automation greatly improves consistency during repeated reviews.

13. Remaining Risks and Improvement Opportunities

- SSH on port 22 remained publicly reachable and could later be restricted by source IP or AWS Security Group design.
- The `ubuntu` account still retained broad administrative capability, which is operationally convenient but high impact.
- The baseline script would benefit from severity scoring and cleaner report path handling.
- Centralized logging and SIEM integration were not yet implemented.
- File integrity monitoring was not yet present.
- Host-level validation did not fully document AWS Security Group rules, which should be included in a cloud-wide review.

14. Recommended Repository Structure

```
project-1-linux-hardening/
├── README.md
├── scripts/
│   └── ec2_security_baseline_check.sh
├── reports/
│   ├── baseline-before-hardening.txt
│   └── baseline-after-hardening.txt
├── docs/
│   ├── threat-model.md
│   ├── hardening-notes.md
│   └── images/
│       └── module-03-security-engineering.png
```

15. Conclusion

Lab 1 and Lab 2 together represent a complete foundational Linux security engineering workflow. The server was first treated as an unknown system that needed to be understood, not assumed secure. Evidence showed a combination of secure defaults and meaningful weaknesses: a public SSH service under active internet scanning, an unnecessary Nginx service exposed on port 80, an additional local user with an active password, an unnecessary SSH feature, and weak ownership or permissions in a security-sensitive directory.

Those findings drove targeted hardening actions. The result was a cleaner and more defensible server posture: reduced attack surface, improved identity control, safer SSH behavior, and stronger host-level hygiene.

The next logical phase is to build on this hardened baseline with logging, abuse detection, and incident response simulation.